

TicTacZero: A TicTacToe Engine

Anish Chandra

University of Michigan-Ann Arbor

anishcha@umich.edu

1. Introduction

The solution to whether a model can autonomously (and independently) learn some optimal strategy in an arbitrary game, without much prior information of the game, has been widely sought out. It is a solution that pushes towards a generality in the current SoTA artificial intelligence.

In 2016, Google Deepmind developed a solution in the form of AlphaGo and later, AlphaGo Zero. These solutions were capable of beating world-renown Go players, and as a result, clearly strategically advanced. This paper pushes to implement the same algorithm for a simpler yet easily tested game, TicTacToe.

2. Related Work

As introduced, this paradigm of game engine was developed primarily by Deepmind in their search to develop an engine for the board game, Go. As such, most of the related work section will revolve around their findings and implementation.

2.1. AlphaGo

Prior to 2016, Go was viewed as an extremely difficult game to develop an engine for, due to the large number of possible moves and the complex evaluation of them. To solve this, many Google Deepmind scientists developed a method to speed up the processing of the search space using Value and Policy Networks to evaluate a board and select optimal moves, respectively. These models were trained using some human generated examples of the game Go, and then by continuing to compete against itself to slowly improve [1].

After AlphaGo, a slightly more optimized model, AlphaGo Zero, was developed that switched to using entirely self-play to train the model, to then feed into the Monte Carlo Tree Search (MCTS). This paper hopes to implement this model in its entirety, for the commonly known board game TicTacToe [2].

3. Method

This section consists of all the design decisions for this engine. This engine implements many of the same as the Al-

phaGo Zero model, however, due to the different nature of TicTacToe, there are some important alterations to the overall paradigm to be discussed. More specifically, there are three main sections: Monte Carlo Tree Search, the Policy and Value Networks, and finally, self-play training.

3.1. Monte Carlo Tree Search

Intuitively, most board games can be devolved down to a simple tree, where the edges are actions and the vertices are the resulting board states. A brute force solution to an engine for these board games is to search for all winning states any arbitrary move can lead to, and pick the move that maximizes the total number of winning states. However, this search operation poorly scales with any increase in action possibilities. Hence when playing complicated games, some optimization is necessary.

The primary optimization that MCTS makes is done by introducing a heuristic. More specifically, it performs a search by running simulations of games by selecting moves using the metric known as the Upper Confidence Bound or the *UCB* score, formulated in Appendix A.

A simulation consists of moves starting from the current state and proceeding by greedily selecting moves using the score until it reaches an undiscovered state of the game. Once it reaches this undiscovered state, it will expand the node (by connecting all possible moves as children), and determine the value of the node. This value is finally then backpropagated through the search path by offsetting each of the nodes' current value by this new value. Note that this is a very rough description of the algorithm, the more formalized pseudocode can be referenced in Appendix B.

3.2. Policy and Value

Recall that the *UCB* score depends on some prior probabilities and values. When a state is undetermined (i.e. neither player has won), its value and priors are determined by 2 neural networks: the Policy and Value. Although these terms are prominently used in RL, in this situation, it can be simplified to be the answers of two questions. The policy network answers how likely each action is to lead to a winning state, and the value network answers how *winnable* a

state of the game is. This information is then fed into MCTS in the form of the *UCB* score to determine how *helpful* a move may be to investigate.

Since TicTacToe is a simple game, the model reflects the same. The model is a standard Feedforward Neural Network with 2 hidden layers, each with 16 neurons and ReLU applied. Finally, there are two output heads: a policy head (9 neurons) and a value head (1 neuron), of which, have applied log-softmax and the sigmoid activation functions (augmented to go from $-1 \rightarrow 1$), respectively.

To determine the loss it can be observed that the policy is effectively a vector of classification probabilities. As such this paper uses a variant of Categorical Cross Entropy loss, defined as the following equation:

$$L_{cross-entropy} = - \sum_i y_i \log(\hat{y}_i) \quad (1)$$

Likewise, the loss for the value function is the Mean Squared Error (MSE) loss, as it's effectively comparing two scalars.

3.3. Self-Play Training

One of the key features of this engine, is that it requires no human generated examples and only relies on playing itself, thereby allowing it to develop its own novel strategies. The algorithm works by repeatedly performing the following:

1. Let the model play itself in *episodes*
2. Store 3-entry tuples with the state of the game, true action probability output by MCTS, and value of who won
3. Train the model on the examples

Note that the MCTS determines the true action probabilities by the number of visits for each child node divided by the number of visits of its parent.

However, there is a case for human generated data as it creates a diversity in the training set of game states. Without this, as soon as the model is shown a game state that it hasn't seen, it will start to make poorly judged moves. In order to suggest the model to diversify its strategies, and potentially lose more often during training, this paper introduces the exploration factor. This factor is used to convert the number of visits a node has, into a more uniform probability distribution, P . More mathematically:

$$P(i) = \frac{x_i^{1/T}}{\sum_j x_j^{1/T}} \quad \text{where,} \quad (2)$$

$$T = \text{exploration factor} \quad (3)$$

$$x_i = i^{th} \text{ action's visit count} \quad (4)$$

Then the model samples from this distribution to output its *likely best* move. Thus, the model is forced to see more diverse boards and learn newer strategies.

4. Experiment

This section describes the more specific implementation process that this engine required. This process is divided into two sections: hyperparameter tuning and the performance of the model against the brute force approach, minimax.

4.1. Hyperparameter Tuning

The hyperparameters for the overall engine was developed to balance the training duration and exploration. In other words, training shouldn't take hours, however, the engine should be given the chance to see varying boards. To do so, the training process involves 30 sets, each consisting of 100 episodes (games), where the model is trained for 50 epochs per set. This sets training time to ~ 10 minutes, while exposing to up to ~ 22500 boards (not unique).

The policy and value networks are trained using the Adam optimizer, set with a learning rate of $5 \cdot 10^{-4}$ to allow a slow and smooth convergence. Finally, the MCTS exploration factor was set to 4, encouraging a more diverse set of training examples. The effects of these choices can be viewed in the loss plot, in Appendix C.

4.2. Competing with Minimax

TicTacToe was primarily chosen due to its nature to be easily solved with a brute force algorithm, minimax. While out of scope, minimax is an algorithm that tries to maximize the reward on its own turn, while minimizing the reward on opponent's turns. Since there are only $3^9 = 19683$ boards (even less due to terminal states), it is reasonable to run this brute force approach. For more details on how the algorithm functions, reference the pseudo code in Appendix D, however, for this paper, assume it as the *perfect* algorithm to benchmark TicTacZero.

When pitted against TicTacZero, the minimax algorithm could only surely win 30% of the time, giving TicTacZero a 70% win-tie rate over 100 games.

5. Conclusion

In conclusion, TicTacZero is a model that was able to learn an optimal strategy, such that it was able to compete with a mathematically correct algorithm. This means that even without human training data, models are able to build themselves up, without the requirement of pre-labeled training data. This paper can safely conclude that the training paradigm provided and built by AlphaGo Zero is applicable across a variety of games/environments. A primary suggestion for future work would be determining a way to teach the model the difference between valid and invalid moves (i.e. determine a way to not require the filtration of invalid moves).

References

- [1] David Silver, Aja Huang, Christopher J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. Mastering the game of go with deep neural networks and tree search. *Nature*, 529:484–503, 2016. [1](#)
- [2] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, Yutian Chen, Timothy Lillicrap, Fan Hui, Laurent Sifre, George van den Driessche, Thore Graepel, and Demis Hassabis. Mastering the game of go without human knowledge. *Nature*, 550(7676):354–359, 2017. [1](#)

TicTacZero: A TicTacToe Engine

Appendix

A. Computing the Upper Confidence Bound Score of a Game State

The *UCB* score is an intuitive equation, it promotes exploration and exploitation. It's derived based on following four relations:

1. Higher prior p (given by the policy) should *raise* the score
2. Higher number of times the node's parent has been visited n_{parent} , should *raise* the score.
3. Higher number of times this node has been visited, n , should *lower* the score
4. Higher ratio of value per visits, V , of this node, should *lower* the score

$$UCB = V + P \quad (5)$$

$$V = \begin{cases} -\sum_{v \in children} \frac{v}{n} & \text{if } n > 0 \\ 0 & \text{otherwise} \end{cases} \quad (6)$$

$$P = \frac{p\sqrt{n_{parent}}}{n + 1} \quad (7)$$

B. Pseudocode for the Monte Carlo Tree Search

Algorithm 1 Monte Carlo Tree Search (MCTS)

```
1: function RUN_SIMULATIONS( $n_{sims}$ )
2:   Initialize root node with the current game state.
3:   for  $n_{sims}$  do
4:     current_node  $\leftarrow$  root
5:     Initialize search_path to empty list
6:     while current_node isn't expanded do
7:       Select the next node based on UCB
8:       Add the selected node to the search_path
9:       if the game ends at the current node then
10:        value  $\leftarrow$  1 if model wins, -1 otherwise
11:       else
12:        Expand the current node and predict value
13:       Add the value to nodes along the search path.
14:   Compute probabilities for each action based on visit counts.
15:   Select the action with the highest visit count as the best move.
16:   return the best action and its probabilities.
```

C. Loss Plot



D. Pseudocode for the Minimax Algorithm

Algorithm 2 Minimax Algorithm for Tic-Tac-Toe

```
1: function SCORE(board, depth)
2:   if PlayerWins(board) then
3:     return 10 - depth
4:   else if OpponentWins(board) then
5:     return -10 + depth
6:   else
7:     return 0
8: function MINIMAX(board, depth, isPlayer)
9:   if GameOver(board) then
10:    return Score(board, depth)
11:   moves  $\leftarrow$  GetValidMoves(board)
12:   Initialize scores as a list
13:   for each move in moves do
14:     Initialize newBoard with move
15:     scores  $\leftarrow$  Minimax(newBoard, depth+1  $\neg$  isPlayer)
16:   if isPlayer then
17:     return move and score that maximizes score
18:   else
19:     return move and score that minimizes score
```
